XO ARENA — INTERNAL DESIGN DOCUMENT

Notification Bus Requirements

Single-pipeline event delivery for all platform notification types

Status Requirements Closed Date April 2026 Audience Engineering

1 Background

XO Arena currently has three separate, uncoordinated notification mechanisms:

Mechanism	Where	What it handles
`notificationService.js`	Backend	Tier upgrades, credit milestones, achievement emails via Resend
`tournamentBridge.js`	Backend	Redis → Socket.io fan-out for tournament lifecycle events
`setInterval` sweep	Tournament service	Auto-start/cancel tournaments past their start time

Each new event type currently requires touching 3–4 files and building a bespoke delivery path. This does not scale.

2 Goals

1. **Single event registry** — one place to define all notification types, delivery targets, persistence rules, and content templates.
2. **Consistent delivery** — every event goes through the same pipeline regardless of source.
3. **Reliable timed events** — replace `setInterval` with a durable periodic dispatcher that survives service restarts.
4. **Offline delivery** — persistent events are held in queue and delivered on reconnect; ephemeral events are socket-only.
5. **User opt-in** — users explicitly choose which notification types they receive and via which channel. No notifications without opt-in except system-critical ones.
6. **Open producer model** — any subsystem or function can publish an event without knowing how delivery works.
7. **Future-ready** — designed to run embedded in the backend now, extractable as a standalone service later without changing producers or consumers.

3 What Already Exists

> These components are kept and extended — not replaced.

- **`UserNotification` table** — `(userId, type, payload, createdAt, deliveredAt, emailedAt)`. Needs `readAt` and `channel` fields added.
- **`notificationService.queueNotification()`** — handles persistence + email. Extend to also emit the socket event.

- `tournamentBridge.js` — refactor to be a thin Redis consumer that calls `dispatch()` rather than containing delivery logic.
- `guideStore.addNotification()` + `NotificationStack` — frontend already handles incoming `guide:notification` socket events. No changes needed.
- **Redis pub/sub** — already the cross-service event backbone. Keep it.

4 Core Design: The Dispatch Function

Any subsystem publishes to the bus by calling a single function:

```
dispatch({
  type:    'tournament.cancelled',    // registered event type
  targets: { cohort: [userId, ...] }, // or { broadcast: true } or { userId }
  payload: { tournamentId, name, reason },
})
```

`dispatch()` is the sole entry point. It:

1. Looks up the event type in the registry to resolve delivery mode, persistence rule, and message template
2. Checks each target user's preferences (skip if opted out, unless the event is system-critical)
3. Routes to the correct delivery path(s) — socket, DB queue, email, SMS — without blocking the caller

> Producers never know how delivery works. Consumers never know where events come from.

5 Event Taxonomy

Delivery Modes

Mode	Description
Broadcast	All connected sockets (e.g. new tournament open, admin announcement)
Cohort	A defined set of user IDs (e.g. all tournament participants)
Personal	Single user (e.g. match ready, challenge received, achievement)

Persistence Rules

Rule	When to use
Ephemeral	Time-sensitive; useless if missed. Socket-only, no DB row.
Persistent	User must receive it eventually. Stored in `UserNotification`, flushed on reconnect.
Persistent + email/SMS	High-importance. Also delivered out-of-band if user is offline and has opted in.

Known Event Types

Event	Mode	Persistence	Notes
`tournament.published`	Broadcast	Ephemeral	New tournament open for registration
`tournament.flash_announced`	Broadcast	Ephemeral	Flash tournament starting soon
`tournament.registration_closing`	Cohort	Persistent	Registration closes in N minutes
`tournament.starting_soon`	Cohort	Persistent	15-min warning before start time
`tournament.started`	Cohort	Persistent	Tournament is now IN_PROGRESS
`tournament.cancelled`	Cohort	Persistent + email	Auto or manual cancel
`tournament.completed`	Cohort	Persistent + email	Final standings and results
`match.ready`	Personal	Persistent	PvP match in tournament ready to play
`match.result`	Personal	Persistent + email	Match outcome recorded
`challenge.received`	Personal	Ephemeral	Another user challenged you 1:1
`challenge.broadcast`	Broadcast	Ephemeral	"Looking for a game" broadcast challenge
`buddy.online`	Personal	Ephemeral	A buddy came online
`game.invite`	Personal	Ephemeral	Invited to a room (legacy; may merge with challenge)
`achievement.tier_upgrade`	Personal	Persistent + email	Already implemented
`achievement.milestone`	Personal	Persistent + email	Already implemented
`admin.announcement`	Broadcast	Persistent	Message from admin to all users
`system.scheduled`	Varies	Varies	Generic timed trigger output

6 Timed Events — Periodic Dispatcher

The current `setInterval` in the tournament sweep is not durable. If the service restarts, in-flight timers are lost.

Approach: DB-backed job table

```
ScheduledJob {
  id           cuid
  type         String        -- e.g. 'tournament.start', 'tournament.warn'
  payload      Json          -- whatever the handler needs
  runAt        DateTime      -- when to execute
  status       Enum          -- PENDING | RUNNING | DONE | FAILED
  attempts     Int default 0
  maxAttempts  Int default 3
  lastError    String?
  createdAt    DateTime
}
```

A periodic dispatcher polls every 30s, atomically claims `PENDING` jobs with `runAt <= now` (sets status to `RUNNING`), executes them via `dispatch()`, then marks `DONE`. Failed jobs increment `attempts`; if `attempts >= maxAttempts` they move to `FAILED` and alert.

Jobs are created by producers when needed:

- Tournament published → schedule `tournament.warn` at `startTime - 15min` and `tournament.start` at `startTime`
- Tournament start time updated → delete old jobs, create new

- Tournament cancelled → delete pending jobs

> The dispatcher emits to the notification bus just like any other producer — it has no special delivery privileges.

7 Buddy List

Users maintain a mutual buddy list. When a user comes online, the bus emits a `buddy.online` ephemeral event to each user who has that player on their list and has opted in to buddy notifications.

> **Mutual requirement:** Both users must accept before either receives buddy notifications about the other. A friend request flow (send → pending → accept/decline) is required. Privacy is a first-class concern — one-way follows are explicitly rejected.

```
BuddyRelationship {
  id          cuid
  userId      String    -- the watcher
  buddyId     String    -- the person being watched
  createdAt   DateTime
  @@unique([userId, buddyId])
}
```

Presence is detected via the existing socket `connect` / `disconnect` events. No polling needed.

8 Challenge System

1:1 Challenge

User A challenges User B directly. Sent as a `challenge.received` personal notification. User B sees it in the Guide panel with Accept/Decline. Auto-expires after **60 seconds** if no response. Ephemeral (no DB row).

Broadcast Challenge

"I'm looking for a game" — sent to all users who have opted in to `challenge.broadcast` notifications. First to accept gets the match. Also ephemeral.

> Both challenge modes are gated by user preferences — users must opt in to receive challenges.

9 User Preferences

Users must **opt in** to receive notifications. Default is off except for system-critical and personal match events.

A dedicated `NotificationPreference` table (rather than columns on `User`) keeps this extensible as new event types are added:

```
NotificationPreference {
  id         cuid
  userId     String
  eventType  String    -- matches event type registry key
  inApp      Boolean   -- show in Guide panel
  email      Boolean   -- send email if offline
  sms        Boolean   -- send SMS if offline (future)
  @@unique([userId, eventType])
}
```

Default Preferences

Event type	In-app default	Email default
`tournament.published`	on	off
`tournament.cancelled`	on	on
`tournament.completed`	on	on
`match.ready`	on	on
`match.result`	on	off
`challenge.received`	on	off
`challenge.broadcast`	**off**	off
`buddy.online`	**off**	off
`admin.announcement`	on	off
`achievement.*`	on	off

> System-critical events (`match.ready` , `tournament.cancelled`) ignore opt-out for in-app delivery but respect email/SMS opt-out.

10 Delivery to Offline Users

When a user reconnects (socket `authenticated` event):

1. Backend queries `UserNotification` for rows where `deliveredAt IS NULL` for that user
2. Emits each as a `guide:notification` socket event in chronological order
3. Marks all as `deliveredAt = now()` in a single update

> Queued notifications are capped (last 20 undelivered) to prevent flooding a user who's been offline for days.

11 Architecture

```
Producer Bus (backend/NotificationService) Delivery ———————————— ——————————————————————————
———————————— Any subsystem —dispatch()—▶ 1. Registry lookup (type → rules) —▶ Socket.io fan-out
Redis consumer —dispatch()—▶ 2. Preference check per target —▶ UserNotification DB Scheduled job
```

```
──dispatch()──▶ 3. Template rendering ──▶ Resend email Admin route ──dispatch()──▶ 4. Route to delivery
paths ──▶ SMS (future) ──▶ Queue flush on reconnect
```

12 Service Extraction (Future)

The bus is designed to run embedded in the backend service initially. To extract it later as a standalone service:

- All producers already communicate via Redis — no code change needed on the producer side
- The bus itself moves to its own process and keeps its Redis subscription and DB connection
- The backend's socket server stays in the backend; the standalone bus calls it via an internal API or shares the Redis channel for socket emission

> **Design constraint:** The bus must never be called synchronously by a producer in a way that creates a hard dependency. All calls are fire-and-forget or async-queued.

13 Observability (Future)

Metric	Description
`notif.dispatched`	Events dispatched per type per minute
`notif.delivered.socket`	Successful socket deliveries
`notif.delivered.email`	Successful email deliveries
`notif.queued.pending`	Undelivered rows in `UserNotification`
`notif.failed`	Failed deliveries (email bounce, socket error)
`scheduler.jobs.pending`	Scheduled jobs waiting to run
`scheduler.jobs.failed`	Jobs that exceeded max attempts
`scheduler.lag_ms`	Time between `runAt` and actual execution

14 Out of Scope

- Push notifications (mobile / browser Push API)
- Notification center / history UI (read/unread inbox)
- Per-tournament notification preferences
- Rate limiting / digest mode
- Webhooks for external integrations
- SMS delivery (architecture supports it; provider TBD)

15 Resolved Design Decisions

Where does the bus live initially?

→ Embedded in the backend service. Tournament service stays a pure Redis producer. The dispatcher module has no direct imports from the rest of the backend — communicates only via `dispatch()` and the DB, making future extraction a deployment change, not a rewrite.

Email sender identity?

→ `noreply@aiarena.callidity.com` via Resend. Driven by the `EMAIL_FROM` environment variable so it can change without a code deploy.

Cohort fan-out at scale?

→ Individual socket rooms (`user:{id}`) is correct for current scale. Revisit at ~10k concurrent users; Redis room broadcasts are the documented upgrade path.

Track `readAt` vs `deliveredAt`?

→ Track both. `deliveredAt` = message reached the device; `readAt` = user saw/dismissed it. Required for a future notification inbox.

Challenge expiry timeout?

→ 60 seconds. After 60s a 1:1 challenge auto-expires if not accepted or declined.

Buddy list — one-way follow or mutual?

→ Mutual. Both users must accept before either receives buddy notifications about the other. A friend request flow (send → pending → accept/decline) is required. Privacy is a first-class concern.

SMS provider?

→ TBD. SMS stays in the preference schema as a future channel; no provider selected, no costs incurred until ready.